

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA SUL-RIO-GRANDENSE

CAMPUS PASSO FUNDO — CURSO DE CIÊNCIA DA COMPUTAÇÃO

Disciplina: Programação Web (PW)

Professora: Jéssica Di Domênico

Alunos: Daniel Silva e Guilherme Vignochi

Tutorial Prático: Drizzle ORM

Mapeamento Objeto-Relacional Moderno e de Alta Performance para Node.js e PostgreSQL

1. Introdução e Contexto

O desenvolvimento de APIs modernas em JavaScript e TypeScript frequentemente lida com o atrito entre dois mundos: o paradigma estruturado/orientado a objetos da aplicação e o modelo relacional dos bancos de dados SQL. Essa divergência é historicamente contornada pelo uso de bibliotecas de **Mapeamento Objeto-Relacional (ORM)**.

Enquanto ferramentas consagradas de gerações anteriores (como Sequelize, TypeORM ou Prisma) agregam abstrações complexas, motores binários externos pesados ou decoradores volumosos, o ecossistema recente demandava uma abordagem minimalista, previsível e sem sobrecarga de execução. É nesse cenário que surge o **Drizzle ORM**.

2. O que é o Drizzle ORM?

O **Drizzle ORM** é uma biblioteca de persistência e mapeamento para ecossistemas TypeScript e JavaScript que adota a filosofia central: *"If you know SQL, you know Drizzle"* (Se você sabe SQL, você sabe Drizzle).

Em vez de esconder a linguagem SQL sob camadas excessivas de regras mágicas, o Drizzle atua como um construtor de consultas estritamente tipado, garantindo:

- **Zero dependências pesadas:** Não utiliza motores intermediários compilados em Rust ou C++.
- **Performance próxima ao SQL nativo:** Apenas constrói e parametriza a consulta antes de passá-la diretamente para o driver de banco (como o `node-postgres`).
- **Inferência estática de tipos:** O esquema das tabelas definido no código gera automaticamente tipos para inserção e seleção sem geração de arquivos em segundo plano.

- **Flexibilidade dupla:** Permite o uso de sintaxe declarativa relacional (*Relational Queries*) ou encadeamento idêntico ao SQL (*SQL-like Query Builder*).

3. Comparativo Técnico de Abordagens

Critério	Drizzle ORM	Prisma	TypeORM
Arquitetura	Modular e ultra-leve	Motor binário em Rust	Decorators e Reflection
Performance	Idêntica ao driver nativo	Média (conversão IPC)	Moderada
Sintaxe	Baseada em SQL	DSL própria (schema.prisma)	Active Record / Data Mapper
Cold Start	Instantâneo (~0ms)	Sensível em Serverless	Razoável
Curva de Aprendizado	Baixa (exige saber SQL)	Baixa/Média	Média/Alta

4. Guia Prático de Utilização

Neste tutorial, integramos o Drizzle ORM a uma base de dados relacional **PostgreSQL**, utilizando como modelo as tabelas da disciplina ([categorias](#) e [produtos](#) do banco [eshoppw](#)).

4.1. Instalação das Dependências

Em um projeto Node.js inicializado ([npm init -y](#)), instale os módulos necessários:

Bash

Dependências de produção: ORM e driver do PostgreSQL

[npm install drizzle-orm pg](#)

Dependência de desenvolvimento: CLI de migrações e painel Studio

[npm install -D drizzle-kit](#)

4.2. Definição do Esquema de Dados ([schema.js](#))

No Drizzle, as tabelas são declaradas diretamente em código JavaScript. Crie o arquivo `schema.js`:

```
import { pgTable, serial, varchar, numeric, integer, boolean, date }
from 'drizzle-orm/pg-core';

// Definição da tabela 'categorias'
export const categorias = pgTable('categorias', {
  codigo: serial('codigo').primaryKey(),
  nome: varchar('nome', { length: 40 }).notNull(),
});

// Definição da tabela 'produtos' com chave estrangeira
export const produtos = pgTable('produtos', {
  codigo: serial('codigo').primaryKey(),
  nome: varchar('nome', { length: 50 }).notNull(),
  descricao: varchar('descricao'),
  quantidadeEstoque: integer('quantidade_estoque').notNull(),
  ativo: boolean('ativo').notNull(),
  valor: numeric('valor', { precision: 12, scale: 2 }).notNull(),
  dataCadastro: date('data_cadastro').notNull(),
  categoriaId: integer('categoria')
    .notNull()
    .references(() => categorias.codigo),
});
```

4.3. Configuração da Conexão com o Banco (db.js)

O Drizzle reutiliza pools de conexão já existentes do pacote nativo `pg`:

```
import { drizzle } from 'drizzle-orm/node-postgres';
import { Pool } from 'pg';
import * as schema from './schema.js';

const pool = new Pool({
  connectionString:
    'postgres://postgres:postgres@localhost:5432/eshoppw',
});

// Instância exportada para executar consultas
export const db = drizzle(pool, { schema });
```

4.4. Operações CRUD na Prática

A. Leitura de Registros (SELECT)

Busca todas as categorias ou filtra por condição específica:

```
import { db } from './db.js';
import { categorias } from './schema.js';
import { eq } from 'drizzle-orm';

// Buscar todas as categorias ordenadas por nome
const todasCategorias = await db
  .select()
  .from(categorias)
  .orderBy(categorias.nome);

// Buscar uma categoria específica por código
const [categoria] = await db
  .select()
  .from(categorias)
  .where(eq(categorias.codigo, 1));
```

B. Inserção de Registros (INSERT)

Insere dados e recupera o registro persistido imediatamente através do comando

```
returning():
JavaScript
const [novaCategoria] = await db
  .insert(categorias)
  .values({
    nome: 'Informática e Hardware'
  })
  .returning();

console.log('Criada com código:', novaCategoria.codigo);
```

C. Atualização de Registros (UPDATE)

Modifica valores com base em critérios explícitos, eliminando risco de atualizações acidentais:

```
const [categoriaAlterada] = await db
  .update(categorias)
  .set({ nome: 'Eletrônicos e Áudio' })
  .where(eq(categorias.codigo, 1))
  .returning();
```

D. Remoção de Registros (DELETE)

Exclui linhas correspondentes à condição:

```
await db
  .delete(categorias)
  .where(eq(categorias.codigo, 2));
```

E. Consultas Relacionais com Junção (INNER JOIN)

Obter produtos acompanhados do nome legível de sua respectiva categoria:

```
import { produtos, categorias } from './schema.js';

const listaProdutos = await db
  .select({
    codigoProduto: produtos.codigo,
    nomeProduto: produtos.nome,
    valor: produtos.valor,
    nomeCategoria: categorias.nome,
  })
  .from(produtos)
  .innerJoin(categorias, eq(produtos.categoriaId, categorias.codigo));
```

5. Ferramental: Drizzle Kit e Drizzle Studio

O ecossistema Drizzle inclui o **Drizzle Kit**, uma ferramenta via linha de comando para automação do banco:

Geração de Migrações SQL:

Cria arquivos `.sql` rastreáveis comparando as alterações do `schema.js` com o estado anterior:

Bash

```
npx drizzle-kit generate
```

Drizzle Studio:

Interface web administrativa embutida que abre localmente no navegador (<http://localhost:4983>), permitindo visualizar, auditar, inserir e testar queries diretamente nas tabelas:

Bash

```
npx drizzle-kit studio
```

6. Conclusão

O **Drizzle ORM** consolida-se como uma das soluções mais modernas e elegantes para desenvolvimento web em Node.js. Ao priorizar a proximidade com o SQL padrão e eliminar

intermediários pesados, ele entrega previsibilidade, código legível e máxima performance em arquiteturas de microsserviços e APIs corporativas.

7. Referências

1. **Drizzle ORM Documentation**. Disponível em: <https://orm.drizzle.team/>. Acesso em: 01 out. 2026.
2. **Node-Postgres (pg)**. PostgreSQL client for Node.js. Disponível em: <https://node-postgres.com/>.
3. **TypeScript Documentation**. Static Type Checking for JavaScript. Disponível em: <https://www.typescriptlang.org/>.

4. **Vídeos recomendados**

<https://www.youtube.com/watch?v=hIYNOiZXQ7Y>

<https://www.youtube.com/watch?v=EwpT466EyP4>

https://www.youtube.com/watch?v=i_mAHOhpBSA